

```
#!/usr/bin/env python3
"""
Multi-Wallet Tracker (Solana)
-----

Tracks any number of wallets (dev wallets, trader wallets, whatever you like)
and sends a clearly formatted Telegram alert whenever one of them:
    - creates a new token   -> shows the token NAME + CA (mint address)
    - buys / swaps a token  -> shows what was paid, what token was bought, its NAM

Config is read from environment variables when available (used on GitHub Actions)
falling back to the hardcoded values below (handy for quick tests on Replit).

Set RUN_ONCE=true (env var) to do a single check-and-exit pass instead of looping
forever – that's what you want on GitHub Actions, since the schedule itself
handles the repeating.
"""

import requests
import time
import json
import os

# ===== CONFIG =====
HELIUS_API_KEY = os.environ.get("HELIUS_API_KEY", "8386d541-876c-4b07-81a4-2ad91c
TELEGRAM_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN", "8752214815:AAFdTsjnx48
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID", "7800248450")

RUN_ONCE = os.environ.get("RUN_ONCE", "false").lower() == "true"

# Add as many wallets as you want. "label" is just for your own reference.
WALLETS = [
    {"label": "Dev wallet 1", "address": "5CEbueQnqlYm2uSSx2xXds3jQAqTlBDnkA59RZo
    # {"label": "Trader wallet 1", "address": "PASTE_ANOTHER_WALLET_HERE"},
]

SOL_MINT = "So1111111111111111111111111111111111111111111111111111111111111111112"

POLL_INTERVAL = 20          # seconds between checks (only used when looping, e.g.
FETCH_LIMIT = 20            # transactions to pull per check
SEEN_FILE = "seen_signatures.json"

# =====

sol_price_cache = {"price": None, "ts": 0}
```

```

def get_sol_price():
    """Live SOL/USD price, cached for 5 minutes so we don't hammer the price API.
    now = time.time()
    if _sol_price_cache["price"] is not None and now - _sol_price_cache["ts"] < 3
        return _sol_price_cache["price"]
    try:
        resp = requests.get(
            "https://api.coingecko.com/api/v3/simple/price",
            params={"ids": "solana", "vs_currencies": "usd"},
            timeout=10,
        )
        price = resp.json()["solana"]["usd"]
        _sol_price_cache["price"] = price
        _sol_price_cache["ts"] = now
        return price
    except Exception:
        return None

```

```

def send_telegram(message):
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    try:
        requests.post(url, data={
            "chat_id": TELEGRAM_CHAT_ID,
            "text": message,
            "disable_web_page_preview": True,
        }, timeout=15)
    except Exception as e:
        print(f"[error] telegram send failed: {e}")

```

```

def load_seen():
    if os.path.exists(SEEN_FILE):
        try:
            with open(SEEN_FILE) as f:
                return json.load(f)
        except Exception:
            return {}
    return {}

```

```

def save_seen(seen):
    trimmed = {addr: sigs[-500:] for addr, sigs in seen.items()}
    with open(SEEN_FILE, "w") as f:
        json.dump(trimmed, f)

```

```

def fetch_transactions(address):
    url = f"https://api.helius.xyz/v0/addresses/{address}/transactions"
    params = {"api-key": HELIUS_API_KEY, "limit": FETCH_LIMIT}
    resp = requests.get(url, params=params, timeout=15)
    resp.raise_for_status()
    return resp.json()

def get_token_name(mint):
    """Look up a token's name + symbol by its mint (CA) using Helius DAS API."""
    if mint == SOL_MINT:
        return "SOL", "SOL"
    url = f"https://mainnet.helius-rpc.com/?api-key={HELIUS_API_KEY}"
    payload = {
        "jsonrpc": "2.0",
        "id": "lookup",
        "method": "getAsset",
        "params": {"id": mint},
    }
    try:
        resp = requests.post(url, json=payload, timeout=15)
        data = resp.json()
        metadata = data.get("result", {}).get("content", {}).get("metadata", {})
        name = metadata.get("name") or "Unknown"
        symbol = metadata.get("symbol") or "?"
        return name, symbol
    except Exception:
        return "Unknown", "?"

def extract_create_ca(tx):
    """For a CREATE transaction, find the mint address of the token that was created
    for transfer in tx.get("tokenTransfers", []) or []:
        mint = transfer.get("mint")
        if mint:
            return mint
    return None

def extract_swap_details(tx):
    """
    For a SWAP transaction, figure out:
    - paid_mint / paid_amount : what the wallet gave up
    - bought_mint / bought_amount : what the wallet received
    Returns a dict, or None if it can't be determined.
    """

```

```

events = tx.get("events", {}) or {}
swap = events.get("swap") or {}

token_inputs = swap.get("tokenInputs", []) or []
token_outputs = swap.get("tokenOutputs", []) or []
native_input = swap.get("nativeInput") # SOL paid, if any
native_output = swap.get("nativeOutput") # SOL received, if any

paid_mint, paid_amount = None, None
bought_mint, bought_amount = None, None

if native_input and float(native_input.get("amount", 0)) > 0:
    paid_mint = SOL_MINT
    paid_amount = int(native_input.get("amount", 0)) / 1e9
elif token_inputs:
    paid_mint = token_inputs[0].get("mint")
    paid_amount = token_inputs[0].get("tokenAmount")

if token_outputs:
    bought_mint = token_outputs[0].get("mint")
    bought_amount = token_outputs[0].get("tokenAmount")
elif native_output and float(native_output.get("amount", 0)) > 0:
    bought_mint = SOL_MINT
    bought_amount = int(native_output.get("amount", 0)) / 1e9

if not paid_mint and not bought_mint:
    return None

return {
    "paid_mint": paid_mint, "paid_amount": paid_amount,
    "bought_mint": bought_mint, "bought_amount": bought_amount,
}

def alert_create(label, address, ca, sig):
    link = f"https://solscan.io/tx/{sig}"
    if ca:
        name, symbol = get_token_name(ca)
        msg = (
            f"
```

```

        f"🆕 NEW COIN CREATED (CA not found – check tx)\n"
        f"Dev wallet: {label} ({address[:4]}...{address[-4:]})\n"
        f"{link}"
    )
    print("\n" + "=" * 60)
    print(msg)
    print("=" * 60)
    send_telegram(msg)

def alert_swap(label, address, details, sig):
    link = f"https://solscan.io/tx/{sig}"
    paid_mint = details["paid_mint"]
    bought_mint = details["bought_mint"]

    paid_name, paid_symbol = (get_token_name(paid_mint) if paid_mint else ("Unkno
    bought_name, bought_symbol = (get_token_name(bought_mint) if bought_mint else

    paid_amount = details.get("paid_amount")
    bought_amount = details.get("bought_amount")

    paid_str = f"{paid_amount:.4f} {paid_symbol}" if paid_amount else paid_symbol
    bought_str = f"{bought_amount:.4f} {bought_symbol}" if bought_amount else bou

    # Add a USD value next to whichever side is SOL, so you know the real size of
    sol_price = get_sol_price()
    if sol_price:
        if paid_mint == SOL_MINT and paid_amount:
            paid_str += f" (~${paid_amount * sol_price:,.2f})"
        if bought_mint == SOL_MINT and bought_amount:
            bought_str += f" (~${bought_amount * sol_price:,.2f})"

    msg = (
        f"💰 SWAP DETECTED\n"
        f"Wallet: {label} ({address[:4]}...{address[-4:]})\n"
        f"Swapped: {paid_str} -> {bought_str}\n"
        f"Bought token: {bought_name} ({bought_symbol})\n"
        f"CA: {bought_mint or '(not found – check tx)'}\n"
        f"{link}"
    )
    print("\n" + "=" * 60)
    print(msg)
    print("=" * 60)
    send_telegram(msg)

def process_wallet(wallet, seen):

```

```

label = wallet["label"]
address = wallet["address"]
seen_sigs = set(seen.get(address, []))
first_run = len(seen_sigs) == 0

try:
    txs = fetch_transactions(address)
except Exception as e:
    print(f"[error] fetch failed for {label}: {e}")
    return seen

new_txs = [tx for tx in txs if tx.get("signature") not in seen_sigs]

for tx in reversed(new_txs): # oldest first
    sig = tx.get("signature", "")
    seen_sigs.add(sig)

    if first_run:
        continue # prime silently, no alert on history

    tx_type = (tx.get("type") or "").upper()
    desc = tx.get("description", "") or ""

    if tx_type == "CREATE" or "created" in desc.lower():
        ca = extract_create_ca(tx)
        alert_create(label, address, ca, sig)
    elif tx_type in ("SWAP", "TOKEN_SWAP") or "swap" in desc.lower():
        details = extract_swap_details(tx)
        if details:
            alert_swap(label, address, details, sig)
        else:
            print(f"[{label}] swap detected but couldn't parse details - {desc}")
    else:
        print(f"[{label}] {tx_type or 'UNKNOWN'} - {desc[:90]}")

if first_run and new_txs:
    print(f"[{label}] primed with {len(seen_sigs)} past transactions.")

seen[address] = list(seen_sigs)
return seen

def run_all_wallets_once(seen):
    for wallet in WALLETS:
        seen = process_wallet(wallet, seen)
        save_seen(seen)
    return seen

```

```

def main():
    if not WALLETS:
        print("No wallets configured - add at least one to the WALLETS list.")
        return

    print(f"Tracking {len(WALLETS)} wallet(s). RUN_ONCE={RUN_ONCE}")
    for w in WALLETS:
        print(f"    - {w['label']}: {w['address']}")
    print()

    seen = load_seen()

    if RUN_ONCE:
        run_all_wallets_once(seen)
        return

    while True:
        seen = run_all_wallets_once(seen)
        time.sleep(POLL_INTERVAL)

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\nStopped.")

```